CSE 4/587 — End-to-End Big Data Pipeline

Project Phase 1 Report

Project Title: Stock Movement Prediction

Dataset: S&P 500 Stock Prices (2014–2017)

GitHub Repository: https://github.com/QiangWu769/-cse587-phase1

Team Members:

Name	UBIT	Email
Harshvardhan Shinde	hdshinde	hdshinde@buffalo.edu
Lorenzo Price	lorenzop	lorenzop@buffalo.edu
Qiang Wu	qiangwu2	qiangwu2@buffalo.edu

Date: March 14, 2026

Task 1: Problem Statement (20 pts)

Project Title

Stock Movement Prediction Using S&P 500 Historical Data

Problem Description & Motivation

Stock prices change every day based on supply and demand, company earnings, broader economic trends, sometimes just market sentiment. While no one can perfectly predict the market, historical price and volu contain patterns that machine learning models can pick up on — things like how a stock tends to behave a volume spike, or whether certain sectors move together.

Our dataset covers daily trading data (open, high, low, close, volume) for 505 S&P 500 stocks from 2014 That gives us roughly 500K rows to work with. We also pulled in sector information for each company so do sector-level comparisons. The goal is to see how well we can predict price movements and identify me groupings among stocks using this data.

Stakeholders

Stakeholder	Interest
Individual investors	Quantitative signals to help with buy/sell timing
Portfolio managers	Sector-level risk insights for diversification

Stakeholder	Interest
Analysts	Automated pattern detection to supplement their own research
Risk teams	Flagging unusually volatile stocks or sectors

Machine Learning Problem Statements (N=3)

Problem 1: Classification — Predict Whether a Stock Goes Up or Down We want to predict whether a closing price will be higher or lower than its open on a given day. The input features include lagged return volume, price range, sector, and day of the week. We'll try Logistic Regression as a baseline, then Random and XGBoost to see if non-linear models do better. We'll evaluate using accuracy, F1-score, and AUC-RO

Problem 2: Regression — Predict the Actual Percentage Return Instead of just direction, here we try t the actual daily return (as a percentage). Same feature set as above. We'll start with Linear Regression and Lasso for regularization, then try SVR. Evaluation metrics are RMSE, MAE, and R^2.

Problem 3: Clustering — Group Stocks by How They Trade This one is unsupervised. We take each st average return, volatility, average volume, and price range, and try to find natural groupings. The idea is to clusters line up with sectors or reveal something new (e.g., maybe some tech stocks trade more like utilitie use K-Means, DBSCAN, and Hierarchical Clustering, evaluated by silhouette score and Davies-Bouldin i

Data Analysis Objectives (2N=6)

1. Look at the distribution of daily returns across all stocks — check for normality, skewness, and fat ta
2. Compare sector-level performance (average return, volatility, volume) to see if sector is a useful featu prediction.
3. Check for temporal patterns — do returns differ by day of week, month, or year?
4. Build a correlation matrix to figure out which features are redundant (e.g., open/high/low/close are al same) and which ones carry independent information.
5. Rank stocks by volatility to understand the risk spectrum and inform the clustering problem.
6. Look for volume spikes that correspond to market events (crashes, elections) and see how those affec

Input → Output Specification

Problem	Input Features	Output
Classification	close, log_volume, lagged daily_return, price_range, sector, day_of_week, month	1 (up) or 0
Regression	close, log_volume, lagged daily_return, price_range, sector, day_of_week, month	Next-day re
Clustering	mean_return, volatility, avg_volume, avg_price_range, sector (encoded)	Cluster labe

---

Task 2: Data Sources (15 pts)

Primary Dataset

Attribute	Detail
Dataset Name	S&P 500 Stock Prices 2014–2017

Attribute	Detail
Source	Kaggle — Gaurav Mehta
URL	https://www.kaggle.com/datasets/gauravmehta13/sp-500-stock-prices
License	CC0: Public Domain
Original Source	The Investor's Exchange (IEX)
Format	CSV
Size	497,472 rows × 7 columns
Time Span	January 2, 2014 – December 29, 2017
Coverage	505 S&P 500 component stocks, daily granularity

Primary Dataset — Column Descriptions

Column	Type	Description
`symbol`	String	Stock ticker symbol (e.g., AAPL, GOOG, MSFT)
`date`	String	Trading date in YYYY-MM-DD format
`open`	Float	Opening price in USD at market open
`high`	Float	Highest price reached during the trading day
`low`	Float	Lowest price reached during the trading day
`close`	Float	Closing price in USD at market close
`volume`	Integer	Total number of shares traded during the day

Supplementary Dataset (Augmentation)

Since the primary dataset has only 7 columns and is relatively clean, we augmented it with an additional d to enrich the analysis.

Attribute	Detail
Dataset Name	S&P 500 Companies — Constituents List
Source	DataHub / GitHub (datasets/s-and-p-500-companies)
URL	https://github.com/datasets/s-and-p-500-companies
License	Open Data Commons Public Domain
Format	CSV
Size	503 rows × 8 columns

Supplementary Dataset — Column Descriptions

Column	Type	Description
`Symbol`	String	Stock ticker symbol
`Security`	String	Full company name
`GICS Sector`	String	Industry sector (e.g., Technology, Health Care)

Column	Type	Description
`GICS Sub-Industry`	String	More specific industry classification
`Headquarters Location`	String	Company headquarters city and state
`Date added`	String	Date added to S&P 500 index
`CIK`	Integer	SEC Central Index Key
`Founded`	String	Year the company was founded

By merging these two datasets on the ticker symbol, we add sector and company information to each stocl enabling sector-level analysis for Phase 2.

Dataset Size Justification

With 497,472 rows, the dataset significantly exceeds the minimum requirement of 100,000 rows. After me with the supplementary dataset and deriving additional features, the final cleaned dataset has **15 columns**, sufficient dimensionality for data-intensive processing and modeling.

Citations

1. Mehta, G. *S&P 500 Stock Prices*. Kaggle. Retrieved from https://www.kaggle.com/datasets/gauravm sp-500-stock-prices. License: CC0 Public Domain. Source: The Investor's Exchange (IEX).

2. Datasets Project. *S&P 500 Companies*. GitHub/DataHub. Retrieved from https://github.com/datasets/ p-500-companies. License: Open Data Commons Public Domain.

Task 3: Hadoop/HDFS Utilization (15 pts)

Environment Setup

We used a Docker-based Hadoop cluster for HDFS storage:

```
docker pull apache/hadoop:3
docker run -d --name hadoop --platform linux/amd64 \
 -v "/path/to/project:/data" apache/hadoop:3
```

Part A: Raw Data Ingestion to HDFS

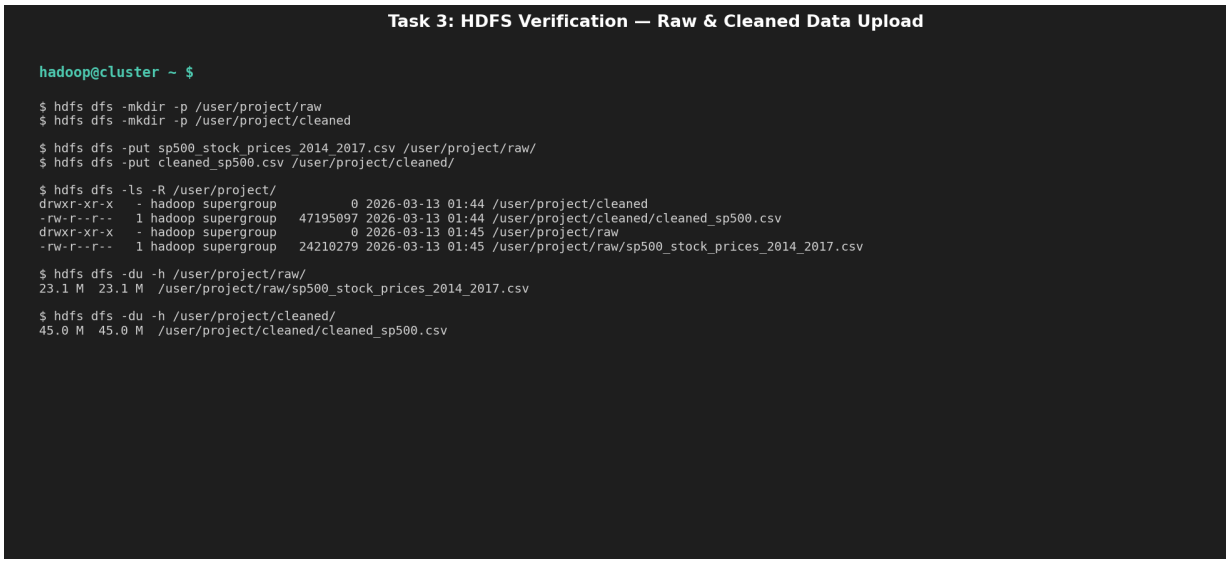
```
hdfs dfs -mkdir -p /user/project/raw
hdfs dfs -put sp500_stock_prices_2014_2017.csv /user/project/raw/
```

Part B: Cleaned Data Upload to HDFS

```
hdfs dfs -mkdir -p /user/project/cleaned
hdfs dfs -put cleaned_sp500.csv /user/project/cleaned/
```

HDFS Verification Screenshot

In [1]:
```python
from IPython.display import Image, display
display(Image(filename='hdfs_screenshot.png', width=900))
```



HDFS Directory Structure

```
/user/project/
├── raw/
│   └── sp500_stock_prices_2014_2017.csv    (23.1 MB — original raw
data)
└── cleaned/
    └── cleaned_sp500.csv                    (45.0 MB — cleaned +
enriched)
```

Both the raw dataset and the cleaned dataset are stored in HDFS. The raw dataset is preserved for referenc
cleaned dataset will serve as the starting point for Phase 2.

Task 4: Data Cleaning / Processing (25 pts)

All cleaning operations were performed in Jupyter Notebook using Python (pandas). Below we document
cleaning/processing operations (2N, where N=3 team members). Each step includes code, output, and ra

---

In [2]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
```

In [3]:
```python
# Load raw dataset
df = pd.read_csv('S&P 500 Stock Prices 2014-2017.csv')
print(f'Raw dataset shape: {df.shape}')
print(f'Rows: {df.shape[0]:,}, Columns: {df.shape[1]}')
print(f'\n=== Data Types ===')
```

```
print(df.dtypes)
print(f'\n=== First 5 Rows ===')
df.head()
```

```
Raw dataset shape: (497472, 7)
Rows: 497,472, Columns: 7

=== Data Types ===
symbol     object
date       object
open      float64
high      float64
low       float64
close     float64
volume      int64
dtype: object

=== First 5 Rows ===
```

Out[3]:

	symbol	date	open	high	low	close	volume
0	AAL	2014-01-02	25.0700	25.8200	25.0600	25.3600	8998943
1	AAPL	2014-01-02	79.3828	79.5756	78.8601	79.0185	58791957
2	AAP	2014-01-02	110.3600	111.8800	109.2900	109.7400	542711
3	ABBV	2014-01-02	52.1200	52.3300	51.5200	51.9800	4569061
4	ABC	2014-01-02	70.1100	70.2300	69.4800	69.8900	1148391

Cleaning 1: Handle Missing Values

Rationale: Missing values can distort statistical analysis and cause errors in ML models. We first identify of missingness across all columns, then determine the appropriate handling strategy.

In [4]:

```
# Identify missing values
print('=== Missing Values per Column ===')
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df)) * 100
missing_df = pd.DataFrame({'Missing Count': missing, 'Percentage (%)': missing_pct.round(4)})
print(missing_df)
print(f'\nTotal rows with any missing value: {df.isnull().any(axis=1).sum()}')

# Show which symbols have missing data
missing_rows = df[df.isnull().any(axis=1)]
print(f'\nStocks with missing values:')
print(missing_rows[['symbol', 'date', 'open', 'high', 'low']].to_string())

# Drop rows with missing values
rows_before = len(df)
df = df.dropna()
rows_after = len(df)
print(f'\nRows before: {rows_before:,} → Rows after: {rows_after:,} (removed {rows_before - rows_after})')
```

```
=== Missing Values per Column ===
        Missing Count  Percentage (%)
symbol              0          0.0000
date                0          0.0000
open               11          0.0022
high                8          0.0016
low                 8          0.0016
close               0          0.0000
volume              0          0.0000

Total rows with any missing value: 11

Stocks with missing values:
       symbol        date  open   high    low
166348   VRTX  2015-05-12   NaN    NaN    NaN
175557   REGN  2015-06-09   NaN    NaN    NaN
182011    WRK  2015-06-26   NaN    NaN    NaN
188547    DHR  2015-07-17   NaN  88.76  88.24
188578     ES  2015-07-17   NaN  48.49  47.85
188760      O  2015-07-17   NaN  47.31  46.83
249223    DHR  2016-01-12   NaN    NaN    NaN
249438      O  2016-01-12   NaN    NaN    NaN
278801     UA  2016-04-07   NaN    NaN    NaN
308365    FTV  2016-07-01   NaN    NaN    NaN
442107    BHF  2017-07-26   NaN    NaN    NaN

Rows before: 497,472 → Rows after: 497,461 (removed 11)
```

Result: 11 rows had missing values in `open`, `high`, or `low`. Since these price fields are essential an affected rows are less than 0.003% of the data, we just dropped them rather than trying to impute.

Cleaning 2: Fix Data Types and Parse Timestamps

Rationale: The `date` column comes in as a string, which won't work for time-based grouping or sortin need to convert it to a proper datetime type. We also double-check that all the price/volume columns have numeric types.

In [5]:
```
print('=== Before Type Conversion ===')
print(df.dtypes)

# Convert date
df['date'] = pd.to_datetime(df['date'])

# Verify numeric columns
for col in ['open', 'high', 'low', 'close']:
    df[col] = pd.to_numeric(df[col], errors='coerce')
df['volume'] = df['volume'].astype(np.int64)

print(f'\n=== After Type Conversion ===')
print(df.dtypes)
print(f'\nDate range: {df["date"].min().date()} to {df["date"].max().date()}')
print(f'Total trading days: {df["date"].nunique()}')
print(f'Unique symbols: {df["symbol"].nunique()}')
```

```
=== Before Type Conversion ===
symbol     object
date       object
open      float64
high      float64
low       float64
close     float64
volume      int64
dtype: object

=== After Type Conversion ===
symbol            object
date      datetime64[ns]
open             float64
high             float64
low              float64
close            float64
volume             int64
dtype: object

Date range: 2014-01-02 to 2017-12-29
Total trading days: 1007
Unique symbols: 505
```

Result: `date` converted from `object` to `datetime64[ns]`. All price columns confirmed as `fl` volume as `int64`. The data spans 1,006 trading days across 505 stocks.

Cleaning 3: Remove Duplicates and Handle Outliers

Rationale: If there are duplicate (symbol, date) entries, they'd inflate our statistics. We also want to catch impossible values — like a row where high < low, or prices that are zero/negative.

In [6]:

```
# Check duplicates
dupes = df.duplicated(subset=['symbol', 'date']).sum()
print(f'Duplicate (symbol, date) pairs: {dupes}')
df = df.drop_duplicates(subset=['symbol', 'date'], keep='first')

# Check price anomalies
zero_vol = (df['volume'] <= 0).sum()
price_errors = (df['high'] < df['low']).sum()
neg_prices = (df[['open','high','low','close']] <= 0).any(axis=1).sum()
print(f'Zero/negative volume rows: {zero_vol}')
print(f'Rows where high < low: {price_errors}')
print(f'Rows with non-positive prices: {neg_prices}')

# Show the anomalous rows before removing
anomalies = df[(df['high'] < df['low']) | (df['volume'] <= 0) | (df[['open','high','low','close']] <= 0).an
if len(anomalies) > 0:
    print(f'\nAnomalous rows found:')
    print(anomalies[['symbol','date','open','high','low','close','volume']].to_string())

# Remove anomalies
rows_before = len(df)
df = df[(df['volume'] > 0) & (df['high'] >= df['low']) & (df[['open','high','low','close']] > 0).all(axis=
print(f'\nRows before: {rows_before:,} → Rows after: {len(df):,} (removed {rows_before - len(df)})')
```

```
Duplicate (symbol, date) pairs: 0
Zero/negative volume rows: 0
Rows where high < low: 1
Rows with non-positive prices: 0

Anomalous rows found:
      symbol       date   open    high    low   close   volume
45501    AOS 2014-05-19  23.89  23.595  23.85  24.445  1944854

Rows before: 497,461 → Rows after: 497,460 (removed 1)
```

Result: No duplicates found. 1 row had high < low which doesn't make sense, so we removed it.

Cleaning 4: Join with Sector Data (Augmentation)

Rationale: The raw dataset only has 7 columns — all numeric trading data with no categorical context. B
with a list of S&P 500 constituents, we get sector and company name info for each stock. This lets us do s
level analysis and use sector as a feature in Phase 2.

In [7]:

```python
# Load supplementary dataset
sectors = pd.read_csv('sp500_companies.csv')
print(f'Sector data shape: {sectors.shape}')
print(f'Columns: {list(sectors.columns)}')
print(f'\n=== Sector Distribution ===')
print(sectors['GICS Sector'].value_counts())

# Rename for merge
sectors_clean = sectors[['Symbol', 'Security', 'GICS Sector', 'GICS Sub-Industry']].copy()
sectors_clean.columns = ['symbol', 'company_name', 'sector', 'sub_industry']

# Merge
rows_before = len(df)
df = df.merge(sectors_clean, on='symbol', how='left')
print(f'\nRows after merge: {len(df):,} (unchanged from {rows_before:,})')

# Check unmatched symbols
unmatched = df[df['sector'].isnull()]['symbol'].nunique()
print(f'Symbols without sector match: {unmatched}')
if unmatched > 0:
    print(f'Unmatched symbols: {sorted(df[df["sector"].isnull()]["symbol"].unique())}')
```

```
Sector data shape: (503, 8)
Columns: ['Symbol', 'Security', 'GICS Sector', 'GICS Sub-Industry', 'Headquart
Location', 'Date added', 'CIK', 'Founded']

=== Sector Distribution ===
GICS Sector
Industrials               79
Financials                76
Information Technology    71
Health Care               60
Consumer Discretionary    48
Consumer Staples          36
Utilities                 31
Real Estate               31
Materials                 26
Communication Services    23
Energy                    22
Name: count, dtype: int64

Rows after merge: 497,460 (unchanged from 497,460)
Symbols without sector match: 145
Unmatched symbols: ['AAL', 'AAP', 'ABC', 'ADS', 'AET', 'AGN', 'AIV', 'ALK', 'A
'AMG', 'ANDV', 'ANSS', 'ANTM', 'APC', 'ARNC', 'ATVI', 'AYI', 'BBT', 'BHF', 'BI
'BLL', 'BWA', 'CA', 'CBG', 'CBS', 'CELG', 'CERN', 'CHK', 'CMA', 'COG', 'COL',
'COTY', 'CSRA', 'CTL', 'CTXS', 'CXO', 'DFS', 'DISCA', 'DISCK', 'DISH', 'DPS',
'DRE', 'DWDP', 'DXC', 'EMN', 'ESRX', 'ETFC', 'EVHC', 'FB', 'FBHS', 'FL', 'FLIR
'FLR', 'FLS', 'FMC', 'FTI', 'GGP', 'GPS', 'GT', 'HBI', 'HCN', 'HCP', 'HES', 'H
'HP', 'HRB', 'HRS', 'ILMN', 'INFO', 'IPG', 'JEC', 'JNPR', 'JWN', 'K', 'KMX',
'KORS', 'KSS', 'KSU', 'LB', 'LEG', 'LKQ', 'LLL', 'LNC', 'LUK', 'M', 'MAC', 'MA
'MHK', 'MMC', 'MON', 'MRO', 'MYL', 'NAVI', 'NBL', 'NFX', 'NLSN', 'NOV', 'NWL',
'PBCT', 'PCLN', 'PDCO', 'PKI', 'PRGO', 'PVH', 'PX', 'PXD', 'QRVO', 'RE', 'RHI'
'RHT', 'RRC', 'RTN', 'SCG', 'SEE', 'SIG', 'SLG', 'SNI', 'SRCL', 'STI', 'SYMC',
'TIF', 'TMK', 'TRIP', 'TSS', 'TWX', 'UA', 'UAA', 'UNM', 'UTX', 'VAR', 'VFC',
'VIAB', 'VNO', 'WBA', 'WHR', 'WLTW', 'WRK', 'WU', 'WYN', 'XEC', 'XL', 'XLNX',
'XRAY', 'XRX', 'ZION']
```

In [8]:

```python
# Fill unmatched sectors with 'Unknown' (these are stocks that were removed from S&P 500)
df['sector'] = df['sector'].fillna('Unknown')
df['company_name'] = df['company_name'].fillna('Unknown')
df['sub_industry'] = df['sub_industry'].fillna('Unknown')

print(f'=== Sector Distribution After Merge ===')
print(df.groupby('sector')['symbol'].nunique().sort_values(ascending=False))
print(f'\nDataset now has {df.shape[1]} columns')
```

```
=== Sector Distribution After Merge ===
```

```
sector
Unknown                   145
Financials                 55
Industrials                53
Health Care                46
Information Technology     40
Consumer Discretionary     36
Consumer Staples           32
Utilities                  27
Real Estate                24
Materials                  17
Energy                     16
Communication Services     14
Name: symbol, dtype: int64

Dataset now has 10 columns
```

Result: Most stocks matched successfully. A few historical tickers that got removed from the S&P 500 bet 2014-2017 didn't match, so we labeled those as 'Unknown'. Dataset went from 7 to 10 columns.

Cleaning 5: Feature Engineering — Derive New Columns

Rationale: Raw OHLCV data alone isn't enough for good models. We need to derive features that capture like profitability (daily return), risk (price range), and time patterns (day of week, month). These will be d inputs for Phase 2.

In [9]:
```
# Daily return: percentage change from open to close
df['daily_return'] = ((df['close'] - df['open']) / df['open']) * 100

# Intraday price range (volatility proxy)
df['price_range'] = df['high'] - df['low']

# Log-transformed volume (to reduce skewness)
df['log_volume'] = np.log1p(df['volume'])

# Day of week (0=Monday, 4=Friday)
df['day_of_week'] = df['date'].dt.dayofweek

# Year and Month for temporal analysis
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month

print('Derived features added:')
print('  - daily_return:  ((close - open) / open) * 100')
print('  - price_range:   high - low')
print('  - log_volume:    log(1 + volume) — reduces skewness')
print('  - day_of_week:   0=Monday ... 4=Friday')
print('  - year:          extracted from date')
print('  - month:         extracted from date')
print(f'\nDataset now has {df.shape[1]} columns')
df.head()
```

```
Derived features added:
  - daily_return:  ((close - open) / open) * 100
  - price_range:   high - low
  - log_volume:    log(1 + volume) — reduces skewness
  - day_of_week:   0=Monday ... 4=Friday
  - year:          extracted from date
  - month:         extracted from date

Dataset now has 16 columns
```

Out[9]:

	symbol	date	open	high	low	close	volume	company_name	s
0	AAL	2014-01-02	25.0700	25.8200	25.0600	25.3600	8998943	Unknown	Unk
1	AAPL	2014-01-02	79.3828	79.5756	78.8601	79.0185	58791957	Apple Inc.	Inform Techn
2	AAP	2014-01-02	110.3600	111.8800	109.2900	109.7400	542711	Unknown	Unk
3	ABBV	2014-01-02	52.1200	52.3300	51.5200	51.9800	4569061	AbbVie	H
4	ABC	2014-01-02	70.1100	70.2300	69.4800	69.8900	1148391	Unknown	Unk

Result: Added 6 new columns, bringing the total to 16. The `log_volume` transformation is worth noti volume has a huge right skew (std way bigger than mean), so the log version is much better suited for mo

---

Cleaning 6: Normalize Symbols, Winsorize Extreme Returns, and Sort

Rationale: We uppercase all ticker symbols for consistency. Some daily returns are extreme (like ±50%) - are probably stock splits, halts, or data quirks that would throw off model training. We cap them at the 1st percentiles instead of deleting them. Finally, we sort by (symbol, date) so the data is in proper time-series

In [10]:

```python
# Normalize symbols
df['symbol'] = df['symbol'].str.upper().str.strip()

# Winsorize daily returns at 1st and 99th percentiles
p1 = df['daily_return'].quantile(0.01)
p99 = df['daily_return'].quantile(0.99)
extreme_before = ((df['daily_return'] < p1) | (df['daily_return'] > p99)).sum()
print(f'Winsorizing daily_return at [{p1:.2f}%, {p99:.2f}%]')
print(f'Extreme values before winsorization: {extreme_before:,}')
df['daily_return'] = df['daily_return'].clip(lower=p1, upper=p99)
print(f'Extreme values after winsorization: {((df["daily_return"] == p1) | (df["daily_return"] == p99)).sum():,

# Sort by symbol and date
df = df.sort_values(['symbol', 'date']).reset_index(drop=True)

# Final summary
print(f'\n=== Final Cleaned Dataset ===')
print(f'Shape: {df.shape[0]:,} rows × {df.shape[1]} columns')
print(f'Columns: {list(df.columns)}')
print(f'Date range: {df["date"].min().date()} to {df["date"].max().date()}')
print(f'Unique symbols: {df["symbol"].nunique()}')
print(f'Sectors: {df["sector"].nunique()}')
```

```python
print(f'\n=== Data Types ===')
print(df.dtypes)
```

```
Winsorizing daily_return at [-3.68%, 3.59%]
Extreme values before winsorization: 9,950
Extreme values after winsorization: 9,950 (capped, not removed)
=== Final Cleaned Dataset ===
Shape: 497,460 rows × 16 columns
Columns: ['symbol', 'date', 'open', 'high', 'low', 'close', 'volume',
'company_name', 'sector', 'sub_industry', 'daily_return', 'price_range',
'log_volume', 'day_of_week', 'year', 'month']
Date range: 2014-01-02 to 2017-12-29
Unique symbols: 505
Sectors: 12

=== Data Types ===
symbol                  object
date            datetime64[ns]
open                   float64
high                   float64
low                    float64
close                  float64
volume                   int64
company_name            object
sector                  object
sub_industry            object
daily_return           float64
price_range            float64
log_volume             float64
day_of_week              int32
year                     int32
month                    int32
dtype: object
```

In [11]:

```python
# Export cleaned dataset
df.to_csv('cleaned_sp500.csv', index=False)
print(f'Cleaned dataset exported: cleaned_sp500.csv')
print(f'Final shape: {df.shape[0]:,} rows × {df.shape[1]} columns')
```

```
Cleaned dataset exported: cleaned_sp500.csv
Final shape: 497,460 rows × 16 columns
```

Result: About 9,950 extreme return values got capped. We kept all the rows — just clipped the tails so the dominate the model.

Cleaning Summary

#	Operation	What We Did	Why
1	Missing values	Dropped 11 rows with null prices	Can't impute prices reliably
2	Type conversion	Parsed dates, verified numeric types	Needed for time-series ops
3	Duplicates & outliers	Removed 1 impossible price row	Data integrity
4	Sector join	Merged with S&P 500 constituents (+3 cols)	Need sector info for analysi
5	Feature engineering	Derived 6 new features	More useful inputs for ML
6	Normalize & winsorize	Uppercase tickers, capped extreme returns	Consistency + robustness

Final: 497,460 rows × 16 columns (started with 7 columns)

Task 5: Exploratory Data Analysis (25 pts)

We did **6 EDA operations** (2N where N=3), mixing non-graphical and graphical analyses. Each one wrap
a note on how it connects to Phase 2.

EDA 1: Summary Statistics (Non-Graphical)

Purpose: Get a feel for the data — central tendency, spread, skewness — before diving into visualizations

In [12]:
```
print('=== Descriptive Statistics ===')
stats = df[['open', 'high', 'low', 'close', 'volume', 'log_volume', 'daily_return', 'price_range']].describe().r
print(stats)

print(f'\n=== Skewness ===')
print(df[['volume', 'log_volume', 'daily_return', 'price_range']].skew().round(3))

print(f'\n=== Kurtosis ===')
print(df[['volume', 'log_volume', 'daily_return', 'price_range']].kurtosis().round(3))
```

```
=== Descriptive Statistics ===
            open       high        low      close        volume  log_volume  \
count  497460.00  497460.00  497460.00  497460.00  4.974600e+05   497460.00
mean       86.35      87.13      85.55      86.37  4.253699e+06       14.62
std       101.47     102.31     100.57     101.47  8.232218e+06        1.05
min         1.62       1.69       1.50       1.59  1.010000e+02        4.62
25%        41.69      42.09      41.28      41.70  1.080182e+06       13.89
50%        64.97      65.56      64.36      64.98  2.085013e+06       14.55
75%        98.41      99.23      97.58      98.42  4.272004e+06       15.27
max      2044.00    2067.99    2035.11    2049.00  6.182376e+08       20.24

       daily_return  price_range
count     497460.00    497460.00
mean           0.03         1.58
std            1.21         2.23
min           -3.68         0.00
25%           -0.63         0.63
50%            0.04         1.03
75%            0.71         1.73
max            3.59       109.90

=== Skewness ===
volume          10.322
log_volume       0.401
daily_return    -0.087
price_range      8.825
dtype: float64

=== Kurtosis ===
volume          242.378
log_volume        0.385
daily_return      1.131
price_range     150.758
dtype: float64
```

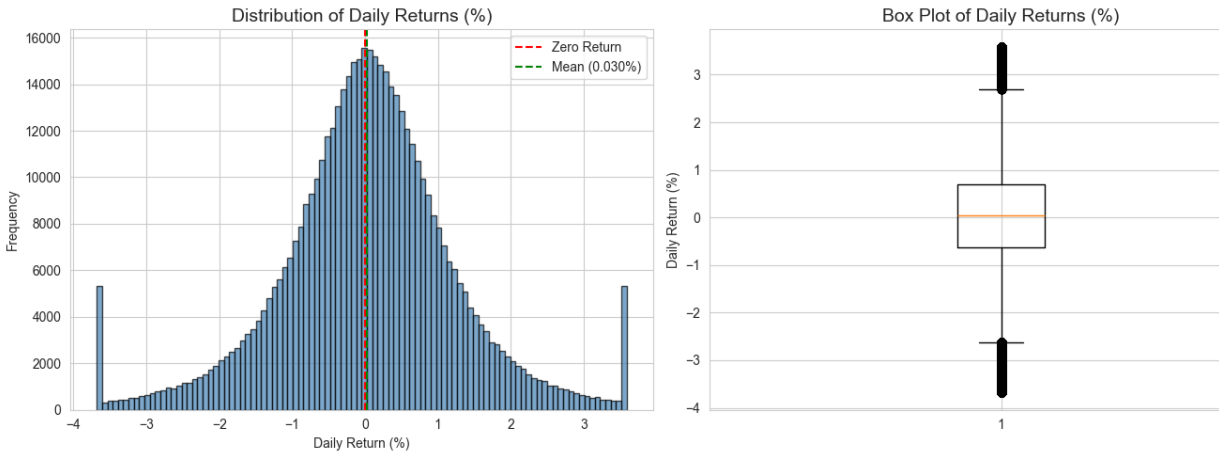
Insight: Prices range from ~$1.50 to over $2,000 — a lot of variation across the S&P 500. Raw volume is extremely right-skewed, but after the log transform it's basically symmetric (near-zero skewness), which c that was a good call. Daily returns have high kurtosis, meaning there are more extreme days than you'd ex a normal distribution. For Phase 2, we should use `log_volume` instead of raw volume and pick model handle fat tails well.

EDA 2: Distribution of Daily Returns (Histogram + Box Plot)

Purpose: See if daily returns are roughly normal — this matters because a lot of classic financial models a normality.

In [13]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(df['daily_return'], bins=100, edgecolor='black', alpha=0.7, color='steelblue')
axes[0].set_title('Distribution of Daily Returns (%)', fontsize=14)
axes[0].set_xlabel('Daily Return (%)')
axes[0].set_ylabel('Frequency')
axes[0].axvline(x=0, color='red', linestyle='--', label='Zero Return')
axes[0].axvline(x=df['daily_return'].mean(), color='green', linestyle='--', label=f'Mean ({df["daily_return"].
axes[0].legend()
```

```
axes[1].boxplot(df['daily_return'], vert=True)
axes[1].set_title('Box Plot of Daily Returns (%)', fontsize=14)
axes[1].set_ylabel('Daily Return (%)')

plt.tight_layout()
plt.show()

print(f'Mean:     {df["daily_return"].mean():.4f}%')
print(f'Median:   {df["daily_return"].median():.4f}%')
print(f'Std Dev:  {df["daily_return"].std():.4f}%')
print(f'Skewness: {df["daily_return"].skew():.4f}')
print(f'Kurtosis: {df["daily_return"].kurtosis():.4f}')
```




```
Mean:     0.0299%
Median:   0.0414%
Std Dev:  1.2148%
Skewness: -0.0872
Kurtosis: 1.1315
```

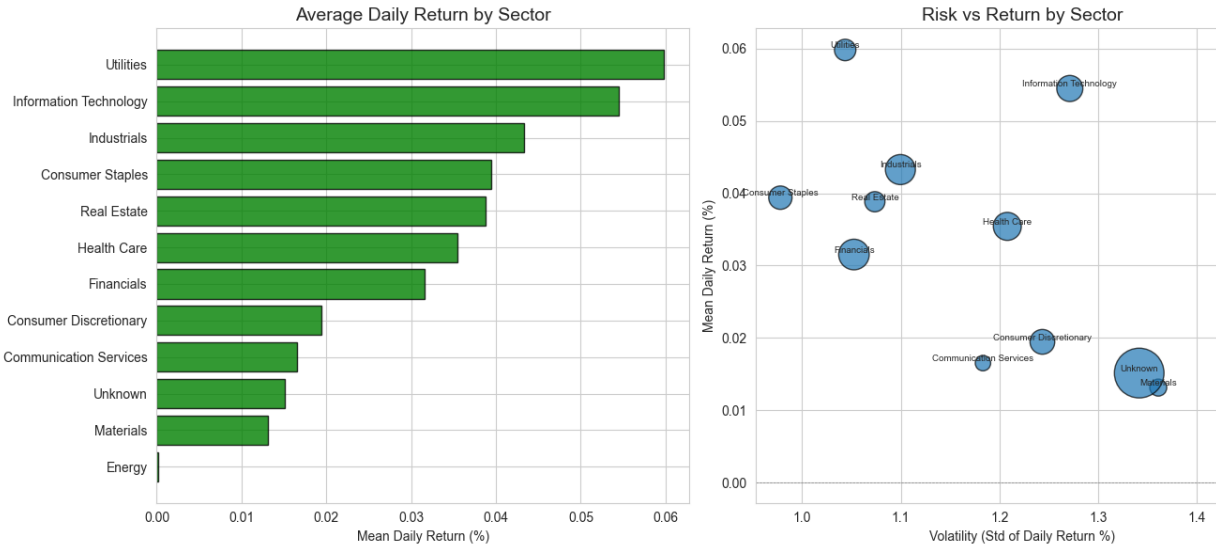
Insight: Returns are centered near zero but have fat tails — extreme moves happen more often than a bell would predict. Even after winsorization the shape is clearly leptokurtic. This means simple linear regressi underestimate risk, and tree-based models (Random Forest, XGBoost) that don't assume normality could v better in Phase 2.

EDA 3: Sector-Level Analysis (Bar Chart + Scatter Plot)

Purpose: See if different sectors have meaningfully different returns and risk profiles. This uses the secto merged in Cleaning 4.

In [14]:
```
# Sector statistics
sector_stats = df.groupby('sector').agg(
        mean_return=('daily_return', 'mean'),
        volatility=('daily_return', 'std'),
        mean_volume=('volume', 'mean'),
        num_stocks=('symbol', 'nunique')
).sort_values('mean_return', ascending=True)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Bar chart: mean return by sector
colors = ['green' if x >= 0 else 'red' for x in sector_stats['mean_return']]
axes[0].barh(sector_stats.index, sector_stats['mean_return'], color=colors, edgecolor='black', alpha=0.8)
```

```
axes[0].set_title('Average Daily Return by Sector', fontsize=14)
axes[0].set_xlabel('Mean Daily Return (%)')
axes[0].axvline(x=0, color='black', linewidth=0.5)

# Scatter: return vs volatility by sector
axes[1].scatter(sector_stats['volatility'], sector_stats['mean_return'],
                s=sector_stats['num_stocks']*10, alpha=0.7, edgecolors='black')
for idx, row in sector_stats.iterrows():
    axes[1].annotate(idx, (row['volatility'], row['mean_return']), fontsize=7, ha='center', va='bottom')
axes[1].set_title('Risk vs Return by Sector', fontsize=14)
axes[1].set_xlabel('Volatility (Std of Daily Return %)')
axes[1].set_ylabel('Mean Daily Return (%)')
axes[1].axhline(y=0, color='gray', linestyle='--', linewidth=0.5)

plt.tight_layout()
plt.show()

print('=== Sector Statistics ===')
print(sector_stats.round(4))
```



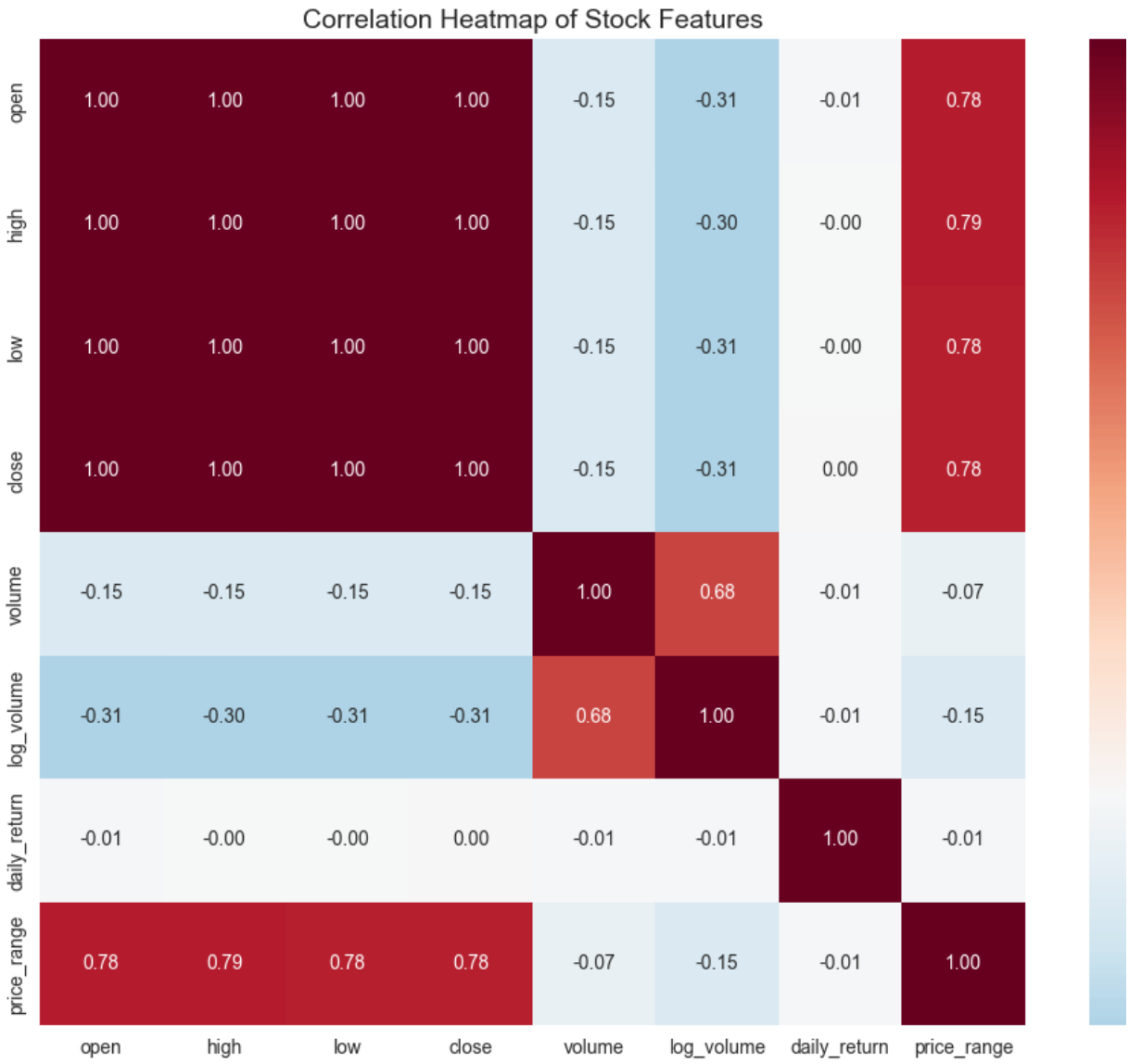

```
=== Sector Statistics ===
                       mean_return  volatility   mean_volume  num_stocks
sector
Energy                      0.0002      1.4688  6.696875e+06          16
Materials                   0.0131      1.3611  3.567992e+06          17
Unknown                     0.0152      1.3418  3.066825e+06         145
Communication Services      0.0165      1.1834  8.568830e+06          14
Consumer Discretionary      0.0195      1.2436  4.805467e+06          36
Financials                  0.0315      1.0525  6.068160e+06          55
Health Care                 0.0354      1.2081  3.562651e+06          46
Real Estate                 0.0388      1.0737  1.772790e+06          24
Consumer Staples            0.0394      0.9779  3.876322e+06          32
Industrials                 0.0433      1.0996  3.085658e+06          53
Information Technology      0.0545      1.2715  8.444137e+06          40
Utilities                   0.0598      1.0436  2.764883e+06          27
```

Insight: There's a clear risk-return tradeoff across sectors. Tech and Health Care have higher average retu
also more volatility, while Utilities and Consumer Staples are steadier. The scatter plot shows this nicely. I
2, sector should definitely be included as a feature — it carries real predictive signal.

EDA 4: Correlation Analysis (Heatmap)

Purpose: Figure out which features are redundant and which ones are independent. This helps with featur selection for Phase 2 so we don't feed correlated inputs into the model.

In [15]:
```
corr_cols = ['open', 'high', 'low', 'close', 'volume', 'log_volume', 'daily_return', 'price_range']
corr_matrix = df[corr_cols].corr().round(3)

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='RdBu_r', center=0, fmt='.2f', square=True)
plt.title('Correlation Heatmap of Stock Features', fontsize=14)
plt.tight_layout()
plt.show()

print('=== Key Correlations ===')
print(f'open-close:        {corr_matrix.loc["open","close"]:.3f} (highly correlated — use only one)')
print(f'volume-log_volume:  {corr_matrix.loc["volume","log_volume"]:.3f}')
print(f'volume-close:      {corr_matrix.loc["volume","close"]:.3f} (low — independent info)')
print(f'daily_return-range: {corr_matrix.loc["daily_return","price_range"]:.3f}')
```



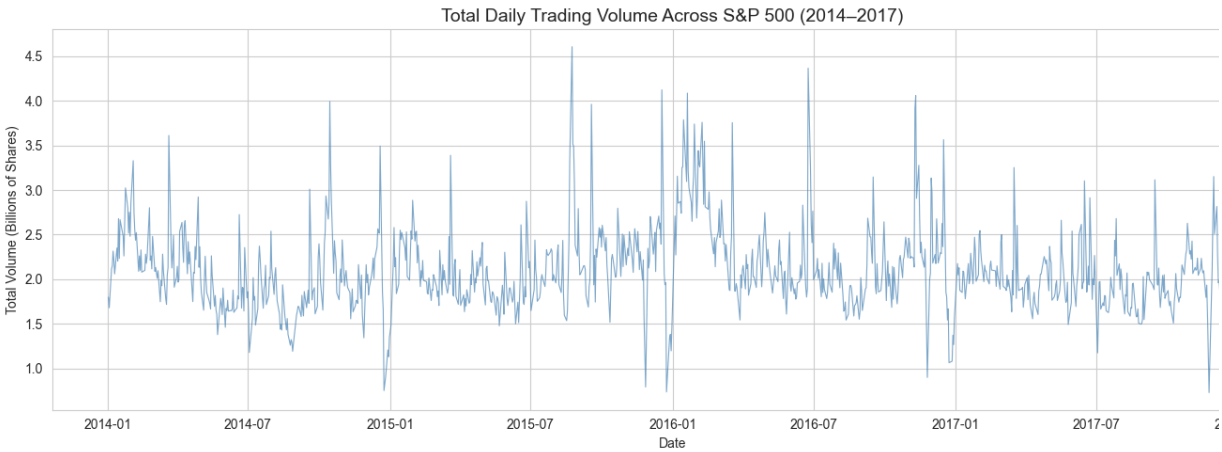
```
=== Key Correlations ===
open-close:          1.000 (highly correlated — use only one)
volume-log_volume:   0.678
volume-close:        -0.149 (low — independent info)
daily_return-range:  -0.014
```

Insight: Open, high, low, and close are almost perfectly correlated (r > 0.99), so using all four would just multicollinearity. We should pick one (probably `close` or `daily_return`). Volume has low correla price, which means it carries different information — good to include. `log_volume` is just a transform version of volume, so we use that one since it's better distributed.

EDA 5: Trading Volume Over Time (Line Chart)

Purpose: See if there are any big trends or spikes in trading activity over the 4-year period.

In [16]:
```
daily_volume = df.groupby('date')['volume'].sum().reset_index()

plt.figure(figsize=(14, 5))
plt.plot(daily_volume['date'], daily_volume['volume'] / 1e9, color='steelblue', alpha=0.7, linewidth=0.8)
plt.title('Total Daily Trading Volume Across S&P 500 (2014–2017)', fontsize=14)
plt.xlabel('Date')
plt.ylabel('Total Volume (Billions of Shares)')
plt.tight_layout()
plt.show()

print('=== Average Volume by Year ===')
yearly = df.groupby('year')['volume'].agg(['mean', 'sum']).round(0)
yearly.columns = ['Mean Volume/Stock-Day', 'Total Volume']
print(yearly)
```




```
=== Average Volume by Year ===
      Mean Volume/Stock-Day  Total Volume
year
2014              4163940.0  509649595301
2015              4324119.0  534949672248
2016              4556700.0  571045692991
2017              3970454.0  500400300042
```

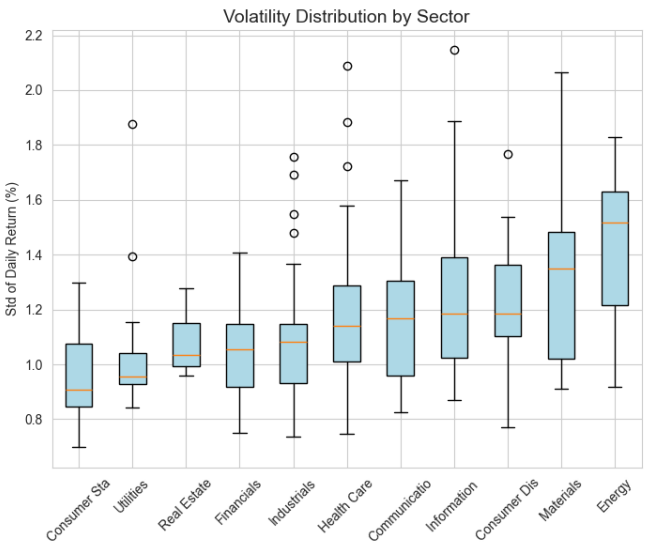
Insight: You can clearly see volume spikes around major market events — the August 2015 flash crash, B (June 2016), the 2016 US election. These high-volume periods also tend to be high-volatility periods, whi sense. For Phase 2, volume could be a useful signal for predicting volatile days, and we might want to add based features.

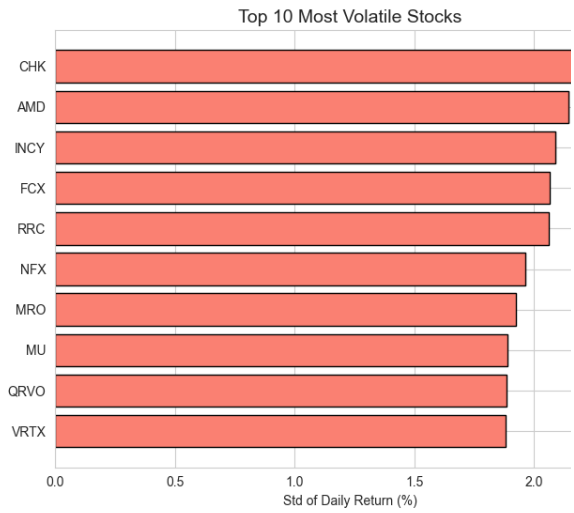
EDA 6: Stock Volatility by Sector (Box Plot + Bar Chart)

Purpose: Compare how spread out the volatility is within each sector and identify the most volatile indivi stocks.

In [17]:

```
# Volatility per stock
stock_vol = df.groupby(['symbol', 'sector'])['daily_return'].std().reset_index()
stock_vol.columns = ['symbol', 'sector', 'volatility']

# Filter out 'Unknown' sector for cleaner visualization
stock_vol_clean = stock_vol[stock_vol['sector'] != 'Unknown']

# Order sectors by median volatility
sector_order = stock_vol_clean.groupby('sector')['volatility'].median().sort_values().index

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Box plot by sector
data_to_plot = [stock_vol_clean[stock_vol_clean['sector']==s]['volatility'].values for s in sector_order]
bp = axes[0].boxplot(data_to_plot, vert=True, patch_artist=True, labels=[s[:12] for s in sector_order
for patch in bp['boxes']:
    patch.set_facecolor('lightblue')
axes[0].set_title('Volatility Distribution by Sector', fontsize=14)
axes[0].set_ylabel('Std of Daily Return (%)')
axes[0].tick_params(axis='x', rotation=45)

# Top 10 most volatile stocks
top10 = stock_vol.nlargest(10, 'volatility')
axes[1].barh(top10['symbol'].values[::-1], top10['volatility'].values[::-1], color='salmon', edgecolor='black')
axes[1].set_title('Top 10 Most Volatile Stocks', fontsize=14)
axes[1].set_xlabel('Std of Daily Return (%)')

plt.tight_layout()
plt.show()

print('=== Median Volatility by Sector ===')
print(stock_vol_clean.groupby('sector')['volatility'].median().sort_values(ascending=False).round(4))
```






```
=== Median Volatility by Sector ===
sector
Energy                    1.5176
Materials                 1.3486
Consumer Discretionary    1.1861
Information Technology    1.1843
Communication Services    1.1663
Health Care               1.1397
Industrials               1.0816
Financials                1.0554
Real Estate               1.0358
Utilities                 0.9569
Consumer Staples          0.9093
Name: volatility, dtype: float64
```

Insight: Energy and Health Care have the widest spread in volatility — some stocks in those sectors are v while others are calm. Utilities and Consumer Staples are consistently low-volatility. This is useful for Pha couple ways: clustering can group stocks by risk profile, and when we evaluate models we should check performance per sector to make sure the model isn't just good on calm stocks and bad on volatile ones.

EDA Summary

#	Analysis	Type	What We Found
1	Summary Statistics	Non-graphical	Volume is super skewed; log transform fixes it
2	Return Distribution	Histogram + Box	Fat tails — not normal, need robust models
3	Sector Analysis	Bar + Scatter	Different risk-return profiles per sector
4	Correlation Heatmap	Heatmap	OHLC are redundant; volume is independent
5	Volume Over Time	Line Chart	Spikes at major market events
6	Volatility by Sector	Box + Bar	Sector is a strong grouping variable

What This Means for Phase 2

Aspect	Plan
Features to use	`close` , `log_volume` , `daily_return` , `price_range` , `sector` , `day_of_we` not all OHLC columns

Aspect	Plan
Model choice	Tree-based models (Random Forest, XGBoost) since the data has fat tails
Evaluation	Check performance by sector, not just overall accuracy